

Bài 2. Lập trình với số và chuỗi

Programming with Numbers and Strings

Trọng tâm bài học

- Biến, hằng, kiểu số và phép gán.
- Biểu thức số học, chuỗi và nhập/xuất dữ liệu.
- Quy trình giải bài toán: tính bằng tay → thuật toán → Python.

Mục tiêu học tập

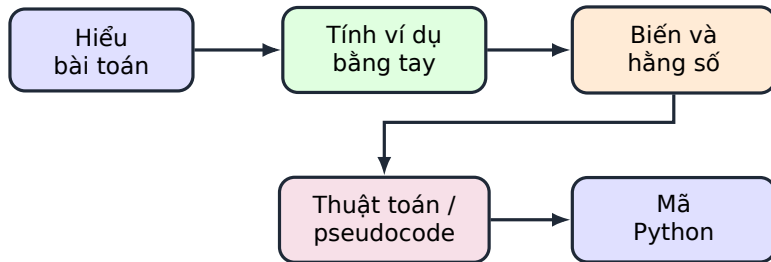
Sau bài học này, sinh viên có thể:

- sử dụng **biến, hằng**, int, float;
- viết biểu thức với +, -, *, /, //, %, **;
- thao tác với chuỗi: nối, lặp, truy cập theo chỉ số và sử dụng phương thức;
- nhận dữ liệu đầu vào, chuyển kiểu và định dạng kết quả đầu ra;
- giải bài toán theo mô hình **Input** → **Process** → **Output** (Đầu vào → Xử lý → Đầu ra).

Cấu trúc bài học

- 1 Biến (Variables)
- 2 Số học (Arithmetic)
- 3 Giải quyết bài toán: trước tiên hãy tính bằng tay
- 4 Chuỗi (Strings)
- 5 Nhập và xuất dữ liệu
- 6 Công cụ bổ sung: SymPy
- 7 Tổng kết và bài tập

Quy trình lập trình trong Chương 2



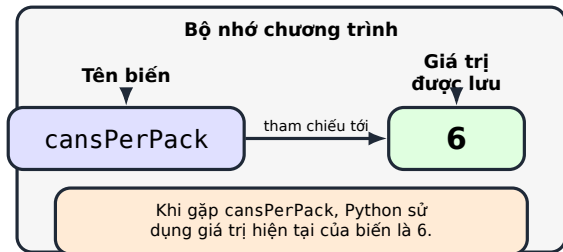
Không nên gõ code ngay; trước hết hãy làm rõ bài toán và cách giải.

Biến là gì?

Khái niệm biến

Một **biến** là tên dùng để tham chiếu đến một giá trị mà chương trình đang lưu trữ.

- **Tên biến** giúp Python xác định giá trị cần truy cập.
- **Giá trị** là dữ liệu hiện được gắn với tên biến.
- Khi sử dụng biến, Python lấy **giá trị hiện tại** gắn với tên biến đó.



Câu lệnh gán

Dạng tổng quát

```
variableName = value
```

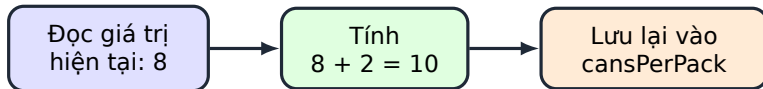
```
cansPerPack = 6  
print(cansPerPack)
```

- Ở lần gán đầu tiên, biến được tạo và nhận giá trị khởi tạo.
- Khi gán lại, giá trị mới sẽ thay thế giá trị cũ.

```
cansPerPack = 6  
cansPerPack = 8  
print(cansPerPack)    # 8
```

Phép gán không phải dấu bằng toán học

```
cansPerPack = cansPerPack + 2
```



Cách hiểu đúng

Trong Python, `=` là toán tử **gán giá trị**, không phải dấu **bằng** trong toán học.

Kiểu dữ liệu số

Kiểu	Ý nghĩa	Ví dụ
int	số nguyên	6, 0, -25
float	số thực	0.355, 1.0, -3.5

- 6 là int; 6.0 là float.
- 1E6 và 2.96E-2 là giá trị dấu phẩy động.
- Kiểu dữ liệu gắn với **giá trị**; tên biến không bị cố định vĩnh viễn với một kiểu.

Tên biến

Quy tắc

- Bắt đầu bằng chữ cái hoặc _.
- Sau đó có thể có chữ, số, _.
- Không chứa khoảng trắng, ?, %, /, ..
- Phân biệt hoa/thường.
- Không dùng từ khóa như `if`, `class`.

Nên dùng tên có ý nghĩa

`canVolume = 0.355`

Tránh

`cv = 0.355` nếu tên quá ngắn và khó hiểu.

Hằng số và chú thích

Hằng số (Constant)

Hằng số là giá trị được quy ước là không thay đổi trong suốt quá trình thực thi chương trình.

```
BOTTLE_VOLUME = 2.0  
MAX_SIZE = 100  
CAN_VOLUME = 0.355  # Liters in a 12-ounce can
```

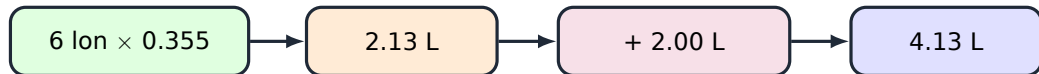
- Giáo trình dùng quy ước tên hằng viết **IN HOA**.
- Chú thích (comment) bắt đầu bằng #, dành cho người đọc và không được Python thực thi.
- Hằng có tên giúp tránh **magic number** – các giá trị số xuất hiện mà không giải thích ý nghĩa.

Ví dụ: tính thể tích nước ngọt

```
CAN_VOLUME = 0.355
BOTTLE_VOLUME = 2.0

cansPerPack = 6
totalVolume = cansPerPack * CAN_VOLUME
print("Six-pack volume:", totalVolume)

totalVolume = totalVolume + BOTTLE_VOLUME
print("Total volume:", totalVolume)
```



Lỗi thường gặp 2.1: dùng biến chưa định nghĩa

Sai

```
canVolume = 12 * literPerOunce  
literPerOunce = 0.0296
```

Lỗi

```
NameError: name 'literPerOunce' is  
not defined
```

- Python thực thi các câu lệnh theo thứ tự từ trên xuống dưới.
- Khi câu lệnh đầu tiên được thực thi, literPerOunce vẫn chưa được định nghĩa.

Đúng

```
literPerOunce = 0.0296  
canVolume = 12 * literPerOunce
```

Tự kiểm tra 2.1: Biến và phép gán

Tự kiểm tra cơ bản

- 1 Khi chạy lần đầu `count = 10`, biến nào được tạo và giá trị là gì?
- 2 Nếu `x = 7` rồi `x = x + 3`, sau đó `x` bằng bao nhiêu?
- 3 `6` và `6.0` khác nhau ở điểm nào?

Tự kiểm tra thêm

- 1 Ký hiệu `=` trong Python dùng để làm gì?
- 2 Tên nào phù hợp cho hằng: `bottleVolume` hay `BOTTLE_VOLUME`?
- 3 Tên nào hợp lệ: `6pack`, `canVolume`, `can volume`?
- 4 Vì sao dùng biến trước khi tạo có thể gây lỗi?

Gợi ý khi làm

Hãy tự trả lời trước, sau đó chạy thử hoặc trao đổi trên lớp để kiểm tra.

Bài tập lập trình ngắn: biến và hằng

Yêu cầu

Viết chương trình tính tổng thể tích của một lốc nước ngọt và một chai lớn.

Dữ liệu cho trước

- Một lốc có 6 lon.
- Mỗi lon có thể tích 0.355 lít.
- Một chai lớn có thể tích 2.0 lít.

Việc cần làm

- Đặt tên hằng phù hợp.
- Tạo biến lưu tổng thể tích của lốc lon.
- Cộng thêm thể tích chai lớn.
- In kết quả ra màn hình.

Các phép toán số học cơ bản

Phép toán	Toán tử	Ví dụ
Cộng	+	$a + b$
Trừ	-	$a - b$
Nhân	*	$a * b$
Chia	/	a / b

Biểu thức (Expression)

Biểu thức là sự kết hợp của giá trị, biến, toán tử, dấu ngoặc và lời gọi hàm để tạo ra một giá trị mới.

$$\frac{a + b}{2} \implies (a + b) / 2$$

Thứ tự ưu tiên toán tử

- 1 Dấu ngoặc
- 2 Lũy thừa
- 3 Nhân và chia
- 4 Cộng và trừ

$a + b / 2$	# not average
$(a + b) / 2$	# average
$10 - 2 - 3$	# $(10 - 2) - 3 = 5$

Lưu ý

Dùng dấu ngoặc để thể hiện rõ thứ tự thực hiện các phép toán và tránh hiểu nhầm.

Lũy thừa

```
2 ** 3      # 8
10 * 2 ** 3 # 80
10 ** 2 ** 3 # 10 ** (2 ** 3)
```

$$b \left(1 + \frac{r}{100}\right)^n \implies b * (1 + r / 100) ** n$$

Điểm cần nhớ

** có ưu tiên cao hơn phép nhân và được tính từ phải sang trái.

Chia lấy phần nguyên và phần dư

// – Floor division

Chia lấy phần nguyên của thương.

7 // 4 # 1

% – Remainder

Lấy phần dư của phép chia.

7 % 4 # 3

```
pennies = 1729
dollars = pennies // 100      # 17
cents = pennies % 100      # 29
```

Một số mẫu với // và %

Với số nguyên dương n:

Biểu thức	Ý nghĩa
$n \% 10$	chữ số cuối
$n // 10$	bỏ chữ số cuối
$n \% 100$	hai chữ số cuối
$n \% 2$	0 nếu chẵn, 1 nếu lẻ

Ứng dụng

Đổi đơn vị, tính tiền thừa, tách chữ số, kiểm tra chẵn/lẻ.

Gọi hàm

Một hàm có thể trả về giá trị để tiếp tục sử dụng trong biểu thức, lưu vào biến hoặc hiển thị ra màn hình.

```
distance = abs(-173)
print(round(7.627, 2))
print(min(7.25, 10.95, 5.95, 6.05))
```

Hàm	Mục đích
abs(x)	trị tuyệt đối
round(x)	làm tròn
round(x,n)	làm tròn đến n chữ số
min(...) / max(...)	nhỏ nhất / lớn nhất

Module math

```
from math import sqrt, pi
```

```
y = sqrt(25)
```

```
area = pi * radius ** 2
```

- `sqrt(x)`: căn bậc hai
- `sin(x)`, `cos(x)`, `tan(x)`
- `log(x)`, `exp(x)`
- `radians(x)`: độ $\rightarrow$ radian
- `degrees(x)`: radian $\rightarrow$ độ
- `trunc(x)`: cắt phần thập phân

Lỗi thường gặp và mẹo viết biểu thức

Sai số làm tròn (roundoff error)

```
price = 4.35  
total = price * 100  
print(total)
```

Số thực dấu phẩy động không thể biểu diễn chính xác mọi giá trị thập phân.

Dấu ngoặc không cân bằng

```
((a + b) * t / 2 * (1 - t)
```

Dấu ngoặc mở và đóng không cân bằng.

Mẹo

Dùng khoảng trắng hợp lý để cấu trúc biểu thức rõ ràng và dễ kiểm tra hơn.

Chủ đề bổ sung 2.1: Import module

Vì sao cần import?

Python chỉ cung cấp trực tiếp một số hàm cơ bản. Các hàm chuyên biệt như `sqrt()`, `sin()`, `cos()` nằm trong các module, chẳng hạn `math`; cần import trước khi sử dụng.

Import module rõ nguồn

```
import math  
x = 25  
y = math.sqrt(x)
```

Import hàm cần dùng

```
from math import sqrt  
x = 25  
y = sqrt(x)
```

Khuyến nghị cho người mới học

Nên dùng `import math` khi muốn mã lệnh rõ nguồn gốc: `math.sqrt(x)` cho biết ngay `sqrt()` thuộc module `math`.

Chủ đề bổ sung 2.2: Gán kết hợp

Ý tưởng

Phép gán kết hợp được dùng khi giá trị mới của một biến được tính trực tiếp từ giá trị hiện tại của chính biến đó.

Dạng đầy đủ

```
total = total + cans  
count = count + 1  
price = price * 2
```

Dạng gán kết hợp

```
total += cans  
count += 1  
price *= 2
```

Cách đọc

`count += 1` nghĩa là lấy giá trị hiện tại của `count`, cộng thêm 1, rồi gán kết quả trở lại cho `count`.

Lưu ý

Biến phải được khởi tạo trước khi sử dụng `+=`, `-=`, `*=` hoặc `/=`.

Chủ đề bổ sung 2.3: Viết biểu thức trên nhiều dòng

Khi nào cần nối dòng?

Khi biểu thức quá dài, có thể tách thành nhiều dòng để mã lệnh dễ đọc hơn. Cách an toàn là xuống dòng khi biểu thức vẫn nằm bên trong cặp dấu ngoặc.

Khó đọc

```
x1 = (-b + sqrt(b ** 2 - 4 * a * c))  
      / (2 * a)
```

Dễ đọc hơn

```
x1 = ((-b + sqrt(b ** 2 - 4 * a * c))  
      / (2 * a))
```

Nguyên tắc thực hành

Giữ các thành phần liên quan trong cùng một nhóm dấu ngoặc và căn dòng tiếp theo để thể hiện rõ biểu thức vẫn còn tiếp tục.

Tự kiểm tra 2.2: Số học và biểu thức

Tự kiểm tra cơ bản

- 1 $10 + 6 / 2$ cho kết quả gì? Vì sao?
- 2 $17 // 5$ và $17 \% 5$ lần lượt là gì?
- 3 Lấy chữ số cuối của số nguyên dương n bằng biểu thức nào?

Tự kiểm tra thêm

- 1 Toán tử nào dùng để tính lũy thừa?
- 2 Vì sao $a + b / 2$ khác $(a + b) / 2$?
- 3 Muốn dùng $\text{sqrt}(x)$ cần import gì?
- 4 Khi nào nên thêm dấu ngoặc trong biểu thức?

Gợi ý khi làm

Dự đoán kết quả trước khi chạy Python; chú ý thứ tự ưu tiên toán tử.

Bài tập lập trình ngắn: đổi giây

Đề bài

Nhập một số giây nguyên dương. Tính số phút đầy đủ và số giây còn lại.

Đầu vào

- Một số nguyên dương biểu diễn tổng số giây.
- Ví dụ dùng để kiểm thử: 135.

Yêu cầu tự làm

- Dùng phép chia lấy phần nguyên `//`.
- Dùng phép lấy phần dư `%`.
- In số phút và số giây còn lại.

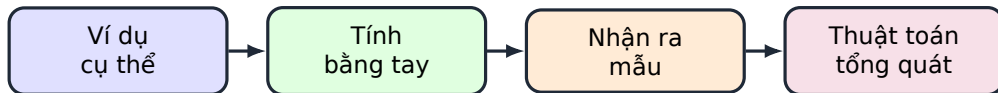
Lưu ý

Không viết hàm `def`; chỉ dùng các câu lệnh cơ bản theo mô hình đầu vào → xử lý → đầu ra.

Trước tiên hãy tính bằng tay

Nguyên tắc

Nếu chưa thể tự giải một ví dụ cụ thể bằng tay, sẽ rất khó viết chương trình tự động hóa phép tính đó một cách đáng tin cậy.



Ví dụ: xếp gạch xen kẽ

Đề bài tóm tắt

- **Đầu vào:** $W = 100$ inch, $T = 5$ inch.
- **Ràng buộc:** gạch đen/trắng xen kẽ; viên đầu và cuối đều màu đen; chỉ dùng số viên nguyên.
- **Đầu ra:** số viên gạch cần dùng và khoảng trống bằng nhau ở hai đầu tường.

Tính tay

- Còn lại sau viên đầu: $100 - 5 = 95$.
- Mỗi cặp trắng-đen: $2 \times 5 = 10$.
- Số cặp hoàn chỉnh: $95 // 10 = 9$.
- Số viên: $1 + 2 \times 9 = 19$.

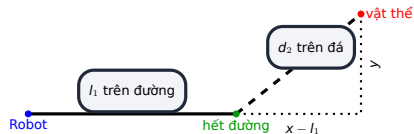
$$\text{khoảng trống mỗi đầu} = \frac{100 - 19 \times 5}{2} = 2.5 \text{ inch}$$

Ví dụ có lời giải 2.1: tính thời gian di chuyển

Đề bài tóm tắt

Robot cần di chuyển tới một vật thể: trước tiên đi trên đường, sau đó đi thẳng qua địa hình đá.

- **Đầu vào:** x, y, l_1, v_1, v_2 .
- **Ràng buộc:** đoạn 1 dài l_1 trên đường; đoạn 2 đi thẳng qua địa hình đá; mỗi địa hình có tốc độ riêng.
- **Đầu ra:** tổng thời gian cần thiết để robot đến vật thể.



Công thức

$$d_2 = \sqrt{(x - l_1)^2 + y^2}$$

$$totalTime = \frac{l_1}{v_1} + \frac{d_2}{v_2}$$

Tự kiểm tra 2.3

- ❶ Vì sao nên tính một ví dụ bằng tay trước khi viết code?
- ❷ Trong ví dụ lát gạch, vì sao `//` hữu ích?
- ❸ Thứ tự nào tốt hơn: viết code trước hay hiểu bài toán và suy ra thuật toán trước?

Chuỗi là gì?

Chuỗi (String)

Chuỗi là một dãy ký tự, có thể gồm chữ cái, chữ số, dấu câu, khoảng trắng và các ký hiệu Unicode.

```
greeting = "Hello"  
print(greeting)
```

```
length = len("World!") # 6
```

- Chuỗi có thể dùng dấu nháy đơn hoặc nháy kép.
- Chuỗi rỗng có độ dài 0: "" hoặc ''.

Nối chuỗi, lặp chuỗi, chuyển đổi

Nối chuỗi

```
firstName = "Harry"  
lastName = "Morgan"  
name = firstName + " " + lastName
```

Lặp chuỗi

```
dashes = "-" * 50
```

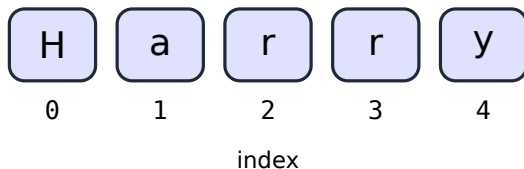
Chuyển đổi

```
id = 1729  
name = "Agent " + str(id)
```

```
id = int("1729")  
price = float("17.29")
```

Chuỗi và chỉ số

Python bắt đầu đếm chỉ số từ 0.



```
name = "Harry"  
first = name[0]  
last = name[len(name) - 1]
```

Lưu ý

Dùng chỉ số ngoài phạm vi hợp lệ sẽ gây exception khi chạy.

Phương thức của chuỗi

Phương thức (Method)

Phương thức là thao tác được gắn với một đối tượng và được gọi thông qua đối tượng đó.

```
name = "John Smith"  
uppercaseName = name.upper()  
lowercaseName = name.lower()  
name2 = name.replace("John", "Jane")
```

Method	Kết quả
s.lower()	chuỗi chữ thường
s.upper()	chuỗi chữ hoa
s.replace(old,new)	chuỗi mới sau thay thế

Unicode và ký tự thoát

Unicode

Python 3 hỗ trợ nhiều hệ chữ viết và ký hiệu.

```
text = "£100"  
print(text[0])
```

- `\n`: xuống dòng.
- `\`: dấu backslash.
- `\"`: dấu nháy kép trong chuỗi.

Ký tự thoát (escape sequence)

```
"You're \"Welcome\""  
"C:\\Temp\\Secret.txt"  
print("*\\n**\\n***")
```

Tự kiểm tra 2.4: Chuỗi

Tự kiểm tra cơ bản

- 1 `len("Hello")` bằng bao nhiêu?
- 2 `"Py" + "thon"` tạo ra chuỗi nào?
- 3 `"-" * 5` tạo ra gì?

Tự kiểm tra thêm

- 1 Với `s = "Python"`, `s[0]` và `s[len(s)-1]` là gì?
- 2 Vì sao `"Agent " + 1729` gây lỗi?
- 3 Muốn nối số 1729 vào chuỗi `"Agent "`, cần dùng hàm nào?

Gợi ý khi làm

Chỉ số chuỗi bắt đầu từ 0; khi nối chuỗi, hai vế phải cùng là chuỗi.

Bài tập lập trình ngắn: xử lý chuỗi

Đề bài

Nhập họ và tên riêng của một người. In tên đầy đủ, độ dài tên đầy đủ và chữ cái đầu của mỗi phần.

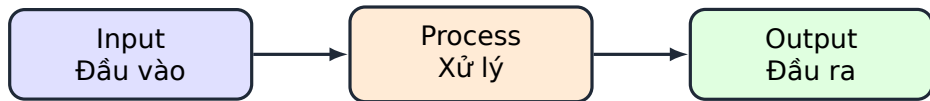
Ví dụ đầu vào

First name: An
Last name: Nguyen

Việc cần làm

- Nối họ và tên bằng dấu cách.
- Dùng `len()` để tính độ dài.
- Dùng chỉ số `[0]` để lấy chữ cái đầu.
- In các kết quả theo nhãn dễ đọc.

Mô hình Đầu vào → Xử lý → Đầu ra

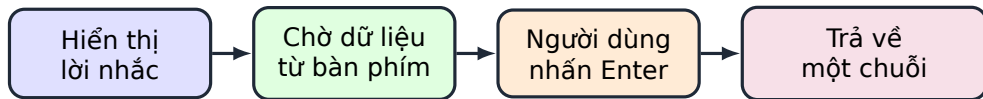


Ý tưởng

Một chương trình thường nhận dữ liệu đầu vào, xử lý dữ liệu theo yêu cầu và tạo ra kết quả đầu ra.

Nhập dữ liệu với `input()`

```
first = input("Enter your first name: ")
```



Quan trọng

`input()` luôn trả về dữ liệu kiểu `str`, ngay cả khi người dùng nhập các ký tự trông giống một số.

Nhập dữ liệu số

Số nguyên

```
userInput = input("Bottles: ")  
bottles = int(userInput)
```

Số thực

```
userInput = input("Price: ")  
price = float(userInput)
```

Có thể viết gọn

```
bottles = int(input("Bottles: "))  
price = float(input("Price: "))
```

Định dạng đầu ra với %

```
price = 1.215962441314554
print("%.2f" % price)      # 1.22
print("%10.2f" % price)    # width 10, 2 decimals
```

Ký hiệu	Ý nghĩa	Ký hiệu	Ý nghĩa
%d	số nguyên	%s	chuỗi
%f	số thực	%9s	chuỗi căn phải
%.2f	2 chữ số thập phân	%-9s	chuỗi căn trái
%7.2f	rộng 7, 2 chữ số	%%	dấu %

Ví dụ: tính giá trên mỗi ounce

```
CANS_PER_PACK = 6

packPrice = float(input("Price for six-pack: "))
canVolume = float(input("Can volume in ounces: "))

packVolume = canVolume * CANS_PER_PACK
pricePerOunce = packPrice / packVolume

print("Price per ounce: %8.2f" % pricePerOunce)
```

Input → Process → Output

Mẹo lập trình 2.4: chuyển kiểu sớm

Ít nên dùng

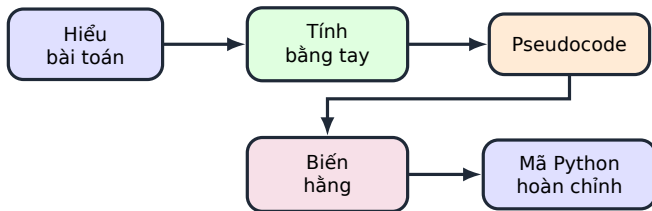
```
unitPrice = input("Price: ")  
price1 = float(unitPrice)  
price2 = 12 * float(unitPrice)
```

Tốt hơn

```
unitPrice = float(input("Price: "))  
price1 = unitPrice  
price2 = 12 * unitPrice
```

Chuyển kiểu ngay sau khi đọc dữ liệu giúp giảm nguy cơ vô tình sử dụng chuỗi trong phép tính số học.

Hướng dẫn 2.1: Viết chương trình đơn giản



Bài toán máy bán hàng - tóm tắt đề bài

- **Đầu vào:** mệnh giá tiền đưa vào *billValue* (đô-la) và giá sản phẩm *itemPrice* (penny).
- **Ràng buộc:** giá là bội số của 25 cent; máy chỉ trả bằng đồng 1 đô-la và quarter.
- **Đầu ra:** số đồng 1 đô-la *dollarCoins* và số quarter *quarters* cần trả lại.

Máy bán hàng: tính bằng tay và pseudocode

Ví dụ cụ thể

- Đầu vào: *billValue* = 5, *itemPrice* = 225 pennies.
- Tiền thừa: $5 \times 100 - 225 = 275$ pennies.
- Đầu ra: 2 đồng 1 đô-la và 3 quarter.

Pseudocode

```
change due = 100 * bill value - item price
dollar coins = change due // 100
change due = change due % 100
quarters = change due // 25
```

Ý tưởng: dùng // để lấy số đồng nguyên và % để giữ lại phần tiền còn dư.

Máy bán hàng: mã Python

```
PENNIES_PER_DOLLAR = 100
PENNIES_PER_QUARTER = 25

billValue = int(input("Enter bill value: "))
itemPrice = int(input("Enter item price in pennies: "))

changeDue = PENNIES_PER_DOLLAR * billValue - itemPrice

dollarCoins = changeDue // PENNIES_PER_DOLLAR
changeDue = changeDue % PENNIES_PER_DOLLAR
quarters = changeDue // PENNIES_PER_QUARTER
```

Ví dụ có lời giải 2.2: tính số tem và tiền thừa

Đề bài tóm tắt

Đầu vào: số đô-la dollars đưa vào máy. **Ràng buộc:** tem hạng nhất giá 49 cent; máy phát tối đa số tem hạng nhất có thể và trả phần dư bằng tem 1 cent. **Đầu ra:** số tem hạng nhất firstClassStamps và số cent còn dư change.

Tính bằng tay với \$1

- Tổng tiền: 100 cent.
- Tem hạng nhất:
 $100 // 49 = 2$.
- Phần dư: $100 - 2 \times 49 = 2$ cent.

Pseudocode

```
total cents = 100 * dollars
first-class stamps =
    total cents // 49
change = total cents % 49
```

Phần tính toán chính

```
FIRST_CLASS_STAMP_PRICE = 49
totalCents = 100 * dollars
firstClassStamps = (
    totalCents
    // FIRST_CLASS_STAMP_PRICE
)
change = totalCents % FIRST_CLASS_STAMP_PRICE
```

Ý tưởng chính: dùng // để xác định số tem hạng nhất tối đa và dùng % để lấy số cent còn dư.

Điện toán & Xã hội 2.2: lỗi trong phần cứng

Lỗi chia số thực trên Pentium

Một lỗi phần cứng từng xuất hiện trong bộ chia số thực của một số CPU Pentium khiến một số phép chia hiếm gặp trả về kết quả không chính xác.

Ví dụ cụ thể

Xét phép chia:

$$\begin{array}{r} 4195835 \\ 3145727 \end{array}$$

- Kết quả đúng: ≈ 1.333820449136 .
- Pentium bị lỗi: ≈ 1.333739068902 .

Nếu nhân thương trở lại với 3145727:

- **Đúng:** 4195835.
- **CPU lỗi:** 4195579.

Bài học

- Kết quả sai không phải lúc nào cũng xuất phát từ mã ứng dụng.
- Nguyên nhân có thể nằm ở chính tầng phần cứng.
- Ngay cả trường hợp hiếm cũng cần được kiểm thử bằng các giá trị chuẩn.
- Với các phép tính quan trọng, nên đối chiếu và xác minh độc lập khi cần thiết.

Tự kiểm tra 2.5: Nhập và xuất dữ liệu

Tự kiểm tra cơ bản

- 1 `input()` luôn trả về kiểu dữ liệu nào?
- 2 Làm thế nào để đọc trực tiếp một số nguyên từ dữ liệu người dùng nhập?
- 3 `%.2f` có ý nghĩa gì?

Tự kiểm tra thêm

- 1 Muốn đọc số nguyên từ bàn phím, kết hợp `input()` với hàm nào?
- 2 Vì sao nên chuyển kiểu dữ liệu số ngay sau khi đọc `input`?
- 3 Toán tử nào hữu ích khi tính tiền thừa bằng đơn vị tiền nguyên?

Gợi ý khi làm

Dữ liệu nhập từ bàn phím cần được chuyển kiểu trước khi dùng trong phép tính số học.

Bài tập lập trình ngắn: hóa đơn đơn giản

Đề bài

Nhập tên sản phẩm, đơn giá và số lượng. Tính tổng tiền và in hóa đơn ngắn.

Đầu vào cần đọc

Product name
Unit price
Quantity

Định dạng đầu ra

Product: <product name>
Unit price: <unit price>
Quantity: <quantity>
Total: <total>

Yêu cầu luyện tập

Kết hợp `input()`, `float()`, `int()`, phép nhân và định dạng số thực với 2 chữ số thập phân.

Công cụ bổ sung 2.1: SymPy

Ý tưởng

SymPy là một package cho toán học ký hiệu, hỗ trợ khai triển biểu thức, giải phương trình, đạo hàm, tích phân, thay thế và vẽ đồ thị.

```
from sympy import *  
  
f = sympify("x ** 2 * sin(x)")  
expand((x - 1) * (x + 1))  
solve(x ** 2 + 2 * x - 8)  
diff(f)  
g = integrate(f)
```

Tổng kết: Biến và số học

Biến

- Biến là vị trí lưu trữ có tên.
- = là phép gán.
- Dùng tên biến có ý nghĩa.
- Dùng hằng có tên để tránh các magic number khó hiểu.

Số học

- /: chia thường.
- //: chia lấy phần nguyên.
- %: phần dư.
- **: lũy thừa.

Tổng kết: Chuỗi và nhập/xuất

Chuỗi

- Chuỗi là dãy ký tự.
- `len()` trả về độ dài.
- `+` nối chuỗi; `*` lặp chuỗi.
- Chỉ số (index) bắt đầu từ 0.

Nhập/Xuất dữ liệu

- `input()` trả về `str`.
- Dùng `int()` hoặc `float()` để chuyển kiểu.
- Dùng toán tử `%` để định dạng đầu ra theo cú pháp của chương.

Câu hỏi tổng kết 1-5

- ❶ Câu lệnh gán làm gì?
- ❷ Giá trị nào có kiểu float: 6, 0, 6.0, -4?
- ❸ `17 // 5` bằng bao nhiêu?
- ❹ `17 % 5` bằng bao nhiêu?
- ❺ `"Py" + "thon"` tạo ra gì?

Câu hỏi tổng kết 6-10

- 6 Với `s = "Hello"`, `s[1]` là gì?
- 7 `input()` trả về kiểu nào?
- 8 Câu lệnh nào đọc số thực: `price = float(input("Price: "))`?
- 9 Vì sao nên dùng hằng có tên?
- 10 Trước khi lập trình phép tính không đơn giản, nên làm gì?

Bài tập ôn tập

Theo dõi các phép gán

```
mystery = 1  
mystery = 1 - 2 * mystery  
mystery = mystery + 1
```

Dịch toán sang Python

- Trung bình của a và b .
- $a^2 + b^2$.
- $\sqrt{x^2 + y^2}$.
- $PV(1 + r/100)^n$.

Bài tập thực hành

- ❶ Đổi kích thước giấy từ inch sang milimét.
- ❷ Đọc một số và in bình phương, lập phương, lũy thừa bậc bốn.
- ❸ Đọc hai số nguyên và in tổng, hiệu, tích, trung bình.
- ❹ Tách một số nguyên dương 5 chữ số thành từng chữ số.
- ❺ Đọc một từ và in ký tự đầu, giữa, cuối.
- ❻ Viết thuật toán tạo chữ viết tắt tên (monogram) cho một tên gồm ba phần.
- ❼ Mở rộng bài máy bán hàng với quarter, dime, nickel.

Thuật ngữ chính

- variable
- assignment
- initialization
- data type
- int / float
- literal
- constant
- magic number

- operator
- expression
- precedence
- floor division
- remainder
- function
- module
- roundoff error

- string
- index
- method
- Unicode
- prompt
- input / output
- format specifier
- canvas

Bài tập tổng hợp ngắn: giá trên mỗi đơn vị

Đề bài

Một gói hàng có nhiều sản phẩm giống nhau. Nhập giá cả gói và số lượng sản phẩm, sau đó tính giá trung bình của một sản phẩm.

Ví dụ input

Pack price: 120000
Quantity: 8

Ví dụ output

Unit price: 15000.00

Yêu cầu

Viết trực tiếp các câu lệnh theo mô hình: Đầu vào → Xử lý → Đầu ra. Chưa dùng def.

Hướng dẫn học tập



Thông điệp

Mục tiêu không phải là ghi nhớ các mẫu cú pháp rời rạc, mà là hình thành quy trình rõ ràng để xây dựng, kiểm thử và cải thiện một chương trình nhỏ.